Maze Escape: Human vs Monster AI Survival Game

Group Number: XX

Member 1 Member 2 Member 3
Member 4 Member 5 Member 6

June 2026

Course Code: SOF106

Course Name: Principles of Artificial Intelligence

Academic Session: 202604

Assessment Title: Final Assessment Group Project

Contents

Abstract

Our project is a survival game called **Maze Escape**. The goal of the project is to show the practical use of artificial intelligence search algorithms and machine learning. The game simulates humans being chased by monsters on a 30×30 toroidal grid map (wrapping edges). The system uses and compares five AI search strategies: **A* pathfinding**, **Greedy Algorithm**, **breadth-first search (BFS)**, **Minimax Algorithm**, and **Simulated Annealing Algorithm**. The maze is generated by a **Genetic Algorithm (GA)**. The GA improves map connectivity, wall density, path diversity, and branch structure through multi-generation evolution.

This project shows how artificial intelligence behaves under limited, competitive, and real-time conditions. It also shows how different search strategies change the difficulty of the game and affect the player experience. The A* algorithm creates the most direct pursuit path. The greedy algorithm gives fast performance with low computing cost. The simulated annealing algorithm adds controlled randomness and makes monster movement less predictable. The genetic algorithm keeps generating connected and navigable maps to maintain gameplay quality. With the addition of machine learning, interceptors can scroll to predict the position of the next three human steps, thus blocking possible escape routes.

1 Introduction

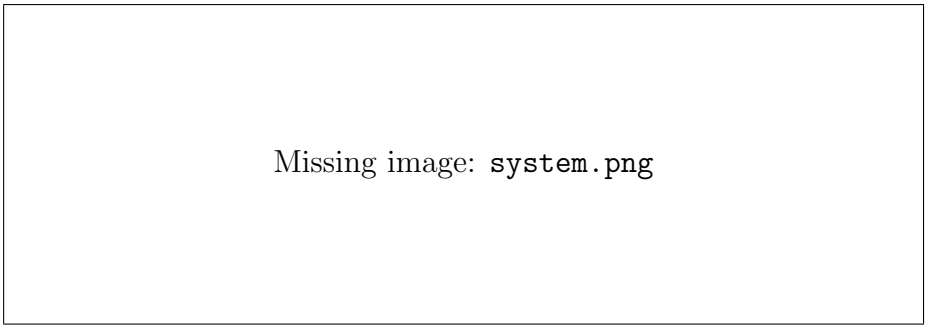

Figure 1: Game System

In the course "Principles of Artificial Intelligence," we studied a variety of search, planning, and optimization algorithms. Based on these core methodologies, our Maze Escape project extends static, single-agent pathfinding into dynamic, multi-dimensional adversarial scenarios. To achieve this transition, we introduce randomly generated maze maps based on genetic algorithms, expand the dimensional space, and a two-way tracking mechanism enhanced by predictive machine learning has been realized.

In order to comprehensively evaluate these algorithms, we have designed two complementary game modes: escape mode and chase mode. In the chase mode, the actions of human characters are driven by evasion algorithms based on Breadth-First Search (BFS). For the core task of predators pursuing prey, the project adopts four distinct intelligent strategies and combined with machine learning-driven interception mechanism:

- **A* Search Algorithm:** In single-monster mode, the algorithm calculates the shortest path to the player. In dual-monster mode, monsters are divided into two

types: direct pursuers and interceptors. The interceptors predict the player's next move and block potential escape routes.

- **Greedy Algorithm:** It calculates a distance field centered at the player's current position, causing monsters to always move toward the nearest neighboring node. This pursuit method is highly efficient and requires low computational resources.
- **Minimax Decision Tree:** This strategy treats the pursuit process as a two-player zero-sum game. The algorithm combines α-β pruning with a transposition table to predict subsequent moves. The monster aims to minimize the player's safety score during the game, while simultaneously simulating the player's efforts to maximize it, thereby enabling efficient pursuit from both perspectives.
- **Simulated Annealing Algorithm:** The algorithm first generates an initial movement path and then randomly adjusts it through repeated iterations. Following the cooling schedule, the system accepts suboptimal moves with a certain probability, making the monster's actions more stochastic and harder to predict.
- **Machine Learning:** Based on spatial characteristics (the relative position of humans relative to monsters, local wall obstacles and the direction of movement of the first two steps), we have trained a random forest model to predict the next move of human players, and predict the position of the next three steps by rolling, so as to achieve effective blockade.

The core environment of our project is a 30×30 toroidal grid map, where the maze layouts are generated by a highly customized Genetic Algorithm (GA). The algorithm maintains a population of candidate maps and evolves them for more than 20 generations. The fitness function balances several critical factors, including strong floor connectivity, approximately 28% wall density, a high number of three-way and four-way junctions, and an average BFS path length close to 20 steps between randomly selected positions. The system also utilizes tournament selection, row-and-column crossover, and bit-flip mutation. These methods effectively assist the algorithm in generating maps with high accessibility, spatial diversity, and appropriate difficulty.

The following sections of this report will introduce the design and specific implementation methods of each component in detail, quantitatively analyze the algorithmic characteristics of each AI strategy along with its impact on gameplay, and reflect on their respective advantages and limitations observed during the development process.

2 Project Objectives

The main objectives of this project are:

- To develop a playable human versus monster survival game.
- To apply artificial intelligence algorithms in a game environment.
- To generate playable maps using a Genetic Algorithm.
- To implement different monster AI strategies.
- To compare the behavior and performance of different algorithms.

3 Methodology

3.1 Genetic Algorithm for Map Generation

The map generation module uses a genetic algorithm to automatically generate the game map. The aim of this part is not just to place walls randomly, but to create a 30×30 map that is connected, playable, and suitable for the chase mechanism in this game. In this project, the map structure directly affects the game balance. If the map is too open, monsters may chase the human too easily. If there are too many walls, the movement of both the human and monsters may become restricted, and they may even be separated into unreachable areas. For this reason, the map generation process needs to balance randomness and playability.

The map is represented as a 30×30 two-dimensional grid. Each cell is stored as a binary value. The value `0` represents walkable ground, while the value `1` represents a wall or obstacle. The human and monsters can only move on walkable cells. The map also uses a wrap-around boundary mechanism, which means that when a character moves out of one side of the map, it will appear from the opposite side. Because of this design, modulus operation is used when calculating neighbouring cells, and Breadth-First Search (BFS) can be applied on this wrap-around map structure for connectivity and path distance checking.

In the genetic algorithm, each map is regarded as a chromosome, while a group of maps forms a population. The initial population is generated randomly, where each cell is assigned as either walkable ground or obstacle according to a fixed wall probability. In this implementation, the target wall ratio is set to around 0.28, which means that about 28% of the map is expected to be walls. This value is used to avoid maps that are either too empty or too crowded.

After the initial maps are generated, each candidate map is evaluated by a fitness function. The fitness function mainly considers four factors: reachability, wall ratio, junction complexity, and path distance. The final fitness score can be expressed as:

$$\text{Fitness} = S_{\text{reachability}} + S_{\text{wall ratio}} + S_{\text{junction}} + S_{\text{path distance}} - P_{\text{dead-end}} \tag{1}$$

Reachability is used to check whether most walkable cells are connected. Wall ratio controls the proportion of walls and compares it with the target value of 0.28. Junction complexity measures the number of branches and intersections, which can provide more possible route choices during the chase. Path distance is used to evaluate whether the average BFS path length in the map is suitable for the survival chase setting. The dead-end penalty is added to reduce the number of cells with very few exits, because too many dead ends may make the map unfair or less playable.

During each generation, the generator calculates the fitness score of every map in the current population. Then, maps with better scores are selected as parents through tournament selection. The crossover operation combines partial structures from two parent maps to generate offspring maps. The mutation operation randomly changes a small number of cells with a low probability, such as changing walkable ground into walls or changing walls into walkable ground. After several generations of selection, crossover, and mutation, the system gradually evolves maps that are more suitable for the game environment.

At the end, the map with the highest fitness score is saved as a text file. The map file uses a space-separated `0` and `1` format, where each line corresponds to one row of the

map. This format is simple, readable, and easy for both Python and C++ game modules to load. Apart from the map grid itself, the system also validates the spawn points of the human and monsters. The spawn points must be located on walkable cells, and the human and monster spawn points cannot overlap. The system also checks the BFS path distance between them. In this implementation, the valid starting distance is set between 10 and 25 BFS steps, so that the monster is not placed too close or too far away from the human at the beginning of the game.

3.2 Human AI Algorithm (BFS)

When the user plays the role of monster, a Breadth-First Search(BFS)-based system is used to control the human character. Generally, the objective of the human character is to keep as far away from the monster as possible. Therefore, every time when the Human AI needs to decide its move direction, we run BFS starting from the monster's current position. This creates a distance map over the whole maze, where each reachable cell stores how many steps the monster would need to reach it. Since the maze is an unweighted grid, BFS gives the shortest path distance correctly.

After building this distance map, the Human AI checks the four possible next positions of the human: up, down, left, and right. It chooses the move with the largest BFS distance from the monster, meaning the human tries to move to the neighboring cell that is farthest away in terms of actual maze path distance.

3.3 Monster AI Algorithms

Several AI algorithms are used for monster movement.

3.3.1 Greedy Search

Greedy Search is the basic pursuit strategy used in the *simple* level. At the beginning of each round, the system runs a Breadth-First Search (BFS) from the current player position to construct a distance field. This distance field stores the real grid-step distance from every reachable cell to the player, while considering walls and wrap-around movement.

After the distance field is computed, each monster checks all cells that it can move to and selects the cell with the shortest distance to the player. This process is repeated in every round. The system also prevents two monsters from occupying the same cell. If a monster is stuck or no valid BFS path to the player exists, Manhattan distance is used as a fallback heuristic, so that the monster can still make forward progress.

Greedy Search is simple and efficient. In each round, the system only needs to perform one BFS computation and a constant number of comparisons for each monster. However, its main limitation is that the monster behaves passively. It only reacts to the player's current position and cannot predict where the player may move next. Against a clever player, the monster may be trapped or forced into inefficient chasing patterns, which increases the time needed to catch the player. Therefore, Greedy Search serves as a well-known baseline strategy and provides the foundation for more advanced pursuit algorithms.

3.3.2 Minimax

Minimax models the chasing process between the monsters and the player as a zero-sum game. The monsters are treated as the minimizing agent, whose goal is to reduce the distance to the player. The player is treated as the maximizing agent, whose goal is to increase the distance from the monsters.

The system builds a depth-limited game tree to represent the possible actions of both sides. Following the round-based structure of the game, the monsters first move two steps, and then the player takes one step. This process is repeated for a fixed number of rounds. In our implementation, the default search depth is set to two in order to keep the computation within the available budget.

Leaf nodes are evaluated using an evaluation function based on the distances between the player and all monsters:

$$\text{Eval} = 4 \cdot \min_i d(\text{Human}, \text{Monster}_i) + \sum_i d(\text{Human}, \text{Monster}_i)$$

In this function, the first term is given a weight of 4 because the nearest monster represents the greatest immediate risk to the player. The summation term encourages all monsters to continue approaching the player, rather than allowing only the closest monster to move effectively. A smaller evaluation score favors the monsters, while a larger score favors the player. If a monster reaches the player's cell, the node returns a large negative score. If the player is caught in fewer steps, the score is made even lower, encouraging the monsters to capture the player as quickly as possible.

Searching the entire game tree is computationally expensive, so several standard optimizations are applied. First, alpha-beta pruning is used to eliminate branches that cannot affect the final decision, thereby reducing the number of nodes that must be searched. Second, a transposition table is used to cache evaluated states. Each state is represented by the player position, the tuple of monster positions, and the current ply index, together with a bound flag. Since the game is played on a wrap-around grid, many states can be reached through different action sequences, so caching helps avoid repeated computation.

Third, move ordering is applied to improve pruning efficiency. Monster moves are ordered by increasing distance to the player, while player moves are ordered by decreasing distance from the monsters. In addition, the best move recorded in the transposition table is checked first when available. A good move ordering allows alpha-beta pruning to cut branches more aggressively and improves the overall search efficiency. After the search is completed, the selected move sequence for the current turn is reconstructed from the transposition table entries and returned through the same movement interface.

Compared with Greedy Search, Minimax provides predictive behavior. Instead of only following the player's current position, the monsters simulate the player's best possible responses and choose moves that can block escape routes. This makes the monsters more strategic and more difficult to avoid. However, the computational cost of Minimax is much higher, and it increases rapidly with the branching factor and search depth. Therefore, the optimizations described above are essential for making the algorithm practical. In this project, Minimax is used for higher difficulty levels where stronger monster intelligence is required.

3.3.3 Simulated Annealing

At the third difficulty level, Simulated Annealing (SA) is used to plan the monster's movement. Unlike A*, SA does not require every intermediate candidate to be the shortest path without walls. Instead, it searches over complete, continuous routes from the monster to the human and assigns large costs to routes that cross walls. This representation makes mutations inexpensive while allowing the algorithm to explore routes that differ substantially from the current one. Since the monster executes only the first two steps of a route before replanning, the objective gives particular importance to the beginning of the path and a wall-aware BFS is retained as a final safety mechanism.

The maze is treated as a torus. For two cells $a = (a_x, a_y)$ and $b = (b_x, b_y)$, their heuristic distance is

$$Dist(a, b) = \min(|a_x - b_x|, 30 - |a_x - b_x|) + \min(|a_y - b_y|, 30 - |a_y - b_y|).$$

The initial state is a random toroidal Manhattan shortest path from the monster to the human. The required horizontal and vertical moves are calculated first and then shuffled. When two wrap-around directions have the same length, one is selected randomly. Consequently, the initial path is continuous and always ends at the human, but it deliberately ignores walls. Wall avoidance is then introduced through mutation and the energy function.

To generate a neighbouring state, the algorithm selects two points on the current path and rebuilds the segment between them. With probability 0.70, the left endpoint is fixed at the monster's current position so that the mutation can change the two steps that will actually be executed. Half of these early mutations rebuild the path all the way to the human; the other half use a right endpoint no more than 30 path positions away. For a non-early mutation, the left endpoint is sampled uniformly from the path and the right endpoint is selected within the same 30-position span.

The replacement segment is constructed by a wall-aware greedy search for at most 80 steps. At each step, it chooses a walkable neighbour with minimum toroidal Manhattan distance to the segment endpoint. Unvisited cells and moves that do not immediately backtrack are preferred, while equal candidates are shuffled to preserve variation. If the greedy search does not reach the right endpoint within its budget, a random toroidal shortest path is appended to close the segment. This closing part may cross walls, but the resulting route must remain continuous, start at the monster, end at the human, and contain no more than 140 cells. A candidate that fails these structural checks is rejected immediately.

For a path $P = (p_0, p_1, \ldots, p_{n-1})$, let h be the human position and $p_{\text{move}} = p_{\min(2,n-1)}$ be the position reached after the move executed in the current turn. The energy minimized by SA is

$$E(P) = |P| + 3Dist(p_{\text{move}}, h) + Wall(P) + R(P).$$

The path-size term favours short routes, while the larger weight on $Dist(p_{\text{move}}, h)$ makes the immediate action maintain pursuit pressure.

The wall cost distinguishes cells near the executable part of the route from those farther in the future:

$$Wall(P) = \sum_{i=1}^{n-1} \begin{cases} 10000, & p_i \text{ is a wall and } i \leq 6, \\ 100, & p_i \text{ is a wall and } i > 6, \\ 0, & p_i \text{ is walkable.} \end{cases}$$

The very large early penalty strongly discourages an illegal current move. The smaller later penalty still guides the search towards a fully walkable path, but permits SA to pass through temporarily poor intermediate states. Because a new route is planned every turn, the first six steps are more important than a distant suffix that may never be executed.

The final term reduces visible oscillation between turns:

$$R(P) = 10I(p_{\text{move}} = p_{\text{prev,start}}),$$

where $p_{\text{prev,start}}$ is the monster position at the beginning of the previous SA move. The game backend records the previous movement and passes it to the C++ planner on the next turn. Thus, returning immediately to the previous starting cell is discouraged without preventing the monster from continuing a useful direction of pursuit.

For a candidate P', an improvement is always accepted. A worse candidate is accepted with probability

$$\Pr(P \to P') = \exp\left(\frac{E(P) - E(P')}{T}\right).$$

This permits broad exploration at high temperature and increasingly selective search as the temperature falls. The implementation also tracks the best accepted route whose first two steps are both adjacent and walkable. That route is preferred to a lower-scoring route that the monster cannot execute safely.

Table 1: Default simulated annealing parameters

Parameter	Value
Candidate trials per temperature	20
Initial temperature	80
Minimum temperature	0.5
Cooling rate	0.94
Immediate-move distance weight	3
Normal wall penalty	100
Wall penalty in the first six steps	10000
Greedy segment-rebuild budget	80 steps
Maximum local segment span	30 path positions
Probability of an early mutation	0.70
Maximum candidate path size	140 cells

With these defaults, the temperatures are $80, 80(0.94), 80(0.94)^2, \ldots$ while $T > 0.5$. This gives 83 temperature levels and $83 \times 20 = 1660$ candidate mutations per call. The parameters can also be overridden from the command line, and a fixed random seed can be supplied for reproducible tests.

After annealing, the monster uses the best accepted route with legal first steps. If no such route is found, a wall-aware toroidal BFS constructs a safe fallback path. The selected route is checked once more before output, ensuring that the monster never finishes its turn inside a wall even though SA is allowed to evaluate wall-crossing candidates during the search.

The current configuration was selected from the obstacle-penalty SA experiment. On five held-out generated maps with five fixed random seeds per map, it caught the human in 24 of 25 games. Its mean planning time was 6.72 ms, the 95th percentile was 10.16 ms,

and the maximum was 12.35 ms. No BFS fallback was required and the selected routes contained no wall visits in these runs. These results do not prove global optimality, but they show that the method is fast enough for interactive play and that the penalty-based search can retain safe movement while producing varied routes.

3.3.4 A* Algorithm

A* is a heuristic graph search algorithm. It finds the shortest path between two nodes by adding the cost of the lined path to the residual cost estimate of reaching the target node. The cost of each node is calculated by the following function:

$$f(n) = g(n) + h(n)$$

In the process of the monster moving to the player in each round, the map grid is regarded as an untitled map, thus calculating the path cost. Due to randomly generated roadblocks, each walkable grid cell has up to four neighbors, which can be reached by moving up, down, left or right. When calculating heuristic functions, we use the Manhattan distance between the current cell and the player's location. In addition, because the map has a ring structure, we will apply modular operations to the row index and column index every time we expand the neighbor. This allows movement to cross the grid boundary. After the algorithm is executed, the core function astar(grid, start, goal) will return the full path from the current position of the monster to the player. With each move of the player, the Astar algorithm will calculate the corresponding optimal path. If there is no path (for example, the player is completely surrounded by a wall), the monster will stay in place.

3.4 Item System

Items are generated after a fixed number of human steps. For example, new items may appear every 10 human steps. Unused items remain on the map. Items can affect the movement and strategy of the game, such as giving the human a speed boost or temporarily stopping the monster.

4 Implementation

The project is divided into several modules. The AI folder stores the algorithm implementations, including Genetic Algorithm, BFS, Greedy Search, A*, Minimax, and Simulated Annealing. The game folder stores the main game logic, including map handling, player movement, monster movement, item logic, collision checking, and scoring.

The Genetic Algorithm module outputs a text file containing the generated map. The game system can then load this map and use it during gameplay.

5 Validation and Verification

Several tests are used to validate the project.

5.1 Map Generation and Spawn Test

The map generation module is tested through several validation steps. First, the program checks whether the generated map size is exactly 30×30 and whether all cell values are strictly either `0` or `1`. This step ensures that the generated map follows the correct format before it is used by the main game program.

Second, BFS-based connectivity checking is used to evaluate whether the walkable cells are sufficiently connected. This helps prevent the map from being divided into isolated regions where the human and monsters cannot reach each other. Since the game uses a wrap-around boundary, BFS is performed based on the same neighbour calculation used in the game movement logic.

Third, the wall density is checked against the target wall ratio. In this project, the target wall ratio is around 0.28. A map with too few walls may make the chase too direct, while a map with too many walls may block movement and reduce playability. Therefore, the wall ratio is included in the fitness function to keep the map at a balanced level.

The spawn points are also validated before the map is used. The human and monsters must start on valid walkable cells, and their positions cannot be the same. The distance between the human and monster spawn points is checked by BFS path distance instead of simple Manhattan distance, because the map contains walls and uses wrap-around boundaries. In the final implementation, the spawn distance is controlled between 10 and 25 BFS steps. This makes the starting positions more suitable for a chase game, because the monster will not catch the human immediately, but it will also not start too far away.

Finally, the generated map is saved as `map/generated_map.txt` and loaded into the main game program. If the main system can correctly load the map and complete human movement, monster chasing, and collision detection, then the map generation module can be considered successfully integrated with the game system. With these checks, the generated map is more than a purely random result, since it satisfies the basic requirements for movement, chasing, and integration with the main game system.

5.2 Boundary Test

The wrap-around boundary is tested by moving characters across the edges of the map. Moving left from column 0 should move the character to column 29. Moving up from row 0 should move the character to row 29.

5.3 Algorithm Comparison

The monster AI algorithms can be compared using capture steps, runtime, and behavior. Greedy Search is simple and fast. A* is more stable for pathfinding. Minimax can predict future movement but requires more computation. Simulated Annealing provides more random behavior.

6 Results and Discussion

The project successfully generates playable maps and supports AI-based monster chasing behavior. The Genetic Algorithm improves map quality compared with purely random

map generation. BFS can support human escape planning. A* provides more reliable chasing behavior than Greedy Search when obstacles exist.

The project also shows that different algorithms have different strengths and weaknesses. Simple algorithms are easier to run but may perform poorly in complex maps. More advanced algorithms can make better decisions but may require more computation.

7 Conclusion

In conclusion, this project demonstrates how artificial intelligence algorithms can be applied in an interactive survival chase game. The Genetic Algorithm is used for map generation, BFS is used for human path searching, and several monster AI algorithms are used for chasing behavior. Through this project, the team gained practical experience in AI search algorithms, optimization, game logic, and system integration.

Future improvements may include better multi-agent monster cooperation, more advanced item effects, improved user interface design, and reinforcement learning-based monster behavior.